

Unidad 1. Introducción a la algorítmica y pseudocódigo

Fundamentos del pensamiento computacional, arquitectura y ciclo de vida del software, evolución de los lenguajes, paradigmas de programación, diagramas de flujo ANSI, pseudocódigo estricto con PSeInt, modularidad y trazas de ejecución paso a paso.



The image shows a dark-themed landing page for a Java course. On the left, there is a green coffee cup icon with steam. Below it, the text reads "CFGS DAM • MÓDULO 0485" and "Programación en Java" in large white and green letters. Underneath, it says "Guía completa desde algoritmia hasta desarrollo modular." and has three buttons: "Java LTS", "IntelliJ IDEA", and "14 Unidades". On the right, there is a code editor window titled "App.java - CFGS DAM" showing a Java code snippet:

```
public class Main {  
    public static void main(String[] args) {  
        // Inicializando entorno 0485  
        Curso dam = new Curso("Java LTS");  
        dam.setIDE("IntelliJ IDEA");  
        dam.cargarModulos(14);  
  
        if (dam.isCompilado()) {  
            System.out.println("¡Hola Mundo!");  
        }  
  
        /* Listo para ejecutar */  
    }  
}
```

MÓDULO PROFESIONAL

Programación (0485) — 1º DAM

DURACIÓN ESTIMADA

8 horas lectivas

RESULTADOS DE APRENDIZAJE

RA1 (Fundamentos del Software y Algorítmica)

ENTORNO OFICIAL

IntelliJ IDEA / PSeInt / Java LTS

¡Te damos la bienvenida al módulo profesional de **Programación (código 0485)** del CFGS de Desarrollo de Aplicaciones Multiplataforma (DAM)!

Antes de abrir el entorno de desarrollo IntelliJ IDEA y sumergirnos en la sintaxis de Java, es imperativo construir la habilidad más determinante en la carrera de cualquier desarrollador de software: el **pensamiento algorítmico**. Un ordenador actual es una máquina prodigiosamente rápida, capaz de realizar miles de millones de operaciones por segundo sin fatigarse, pero carece por completo de intuición, criterio o sentido común. La computadora hará con absoluta fidelidad y a la velocidad de la luz exactamente lo que le indiques; si diseñas una solución lógica defectuosa, la máquina ejecutará tu error sin pestañear.

En esta primera unidad didáctica aprenderás a pensar de manera estructurada, analítica y modular, transformando problemas del mundo real en algoritmos rigurosos, verificables e independientes del lenguaje final.

Recursos y Descargas en PDF de la Unidad 1

Puedes estudiar esta unidad tanto en la plataforma web como en formato documento digital/impreso:

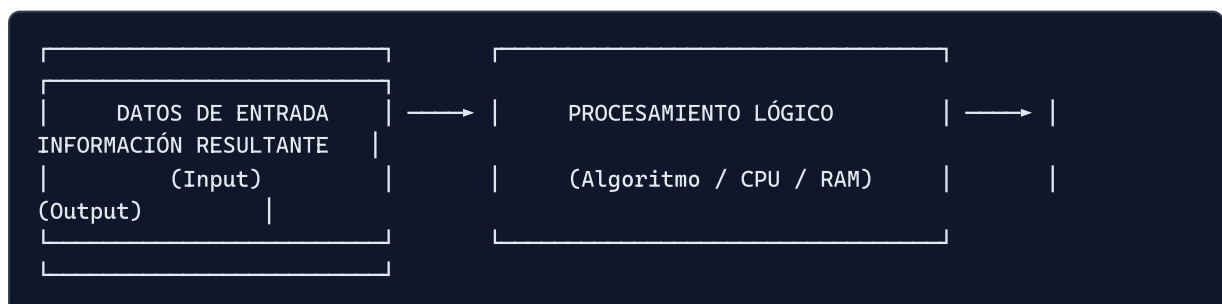
-  [Descargar Teoría Completa de la Unidad 1 en PDF](#) (25 páginas)
-  **Boletín Inicial:** [Descargar Propuestos \(PDF\)](#) · [Descargar Resueltos \(PDF\)](#)
-  **Boletín Intermedio:** [Descargar Propuestos \(PDF\)](#) · [Descargar Resueltos \(PDF\)](#)
-  **Retos y Extras:** [Descargar Retos \(PDF\)](#)

1. Fundamentos: Programas, Procesos y Tipología del Software

1.1 El Enfoque de Caja Negra (*Black Box*)

La Real Academia Española define un programa como un *“conjunto de instrucciones que permite a un ordenador realizar funciones diversas”*. Dicho de forma más intuitiva: un programa es una **receta** que un ordenador sigue al pie de la letra, sin criterio propio. Hace exactamente lo que le indicas, ni más ni menos (y esa fidelidad ciega es la causa de que los errores o *bugs* existan).

Desde el punto de vista de la ingeniería informática, cualquier programa puede modelarse conceptualmente como una **caja negra**: un sistema cerrado del cual nos interesa qué entra, qué sale y qué transformaciones realiza, aislando temporalmente la complejidad interna.



1. **Entrada (Input):** Datos iniciales no procesados procedentes de periféricos (teclado, ratón), medios de almacenamiento secundario (ficheros, bases de datos) o interfaces de red (llamadas a APIs, sockets).
2. **Procesamiento:** Secuencia lógica y ordenada de instrucciones que transforman, comparan, calculan o combinan los datos recibidos haciendo uso de la memoria principal (RAM) y la Unidad Central de Procesamiento (CPU).

3. **Salida (Output):** Información útil estructurada y comunicada al usuario a través de periféricos de salida (pantalla, altavoces), persistida en disco o transmitida telemáticamente a otro sistema.

1.2 Distinción Conceptual Rigurosa

En el argot profesional a menudo se confunden términos que tienen matices técnicos muy concretos:

- **Algoritmo:** Procedimiento computacional paso a paso, formal y abstracto, que toma un conjunto de valores como entrada y produce un resultado como salida. Es completamente independiente del hardware y del lenguaje de programación.
- **Programa:** Codificación concreta de uno o varios algoritmos en un lenguaje de programación formal (Java, C, Python), almacenada como texto fuente legible o compilada en disco.
- **Proceso:** Instancia de un programa en ejecución en la memoria RAM del ordenador, con su propio espacio de direcciones, contador de programa, registros y recursos concedidos por el Sistema Operativo.
- **Aplicación / Software:** Sistema informático integral formado por múltiples programas, bibliotecas (*frameworks* / *APIs*), interfaces gráficas o de consola, ficheros de configuración, recursos multimedia y documentación técnica.

1.3 Clasificación del Software según su Entorno de Ejecución

En el ecosistema profesional actual, el software que desarrollamos adopta diversas arquitecturas según las necesidades del usuario final:

Tipo de Aplicación	Descripción y Características	Ejemplos Habituales
Aplicaciones de Consola (CLI / Terminal)	Interactúan exclusivamente en modo texto a través de la terminal o símbolo del sistema, sin interfaz gráfica de ventanas. Es el formato idóneo para iniciarse en la programación y centrarse en la lógica pura sin distracciones visuales (el punto de partida de este curso).	Comandos del sistema operativo, scripts de administración, utilidades de diagnóstico.
Aplicaciones de Escritorio (Desktop / GUI)	Se instalan y ejecutan localmente en el sistema operativo del usuario (Windows, Linux, macOS) ofreciendo ventanas, botones y menús gráficos interactivos. Tienen acceso directo al hardware, máxima fluidez y pueden funcionar desconectadas de la red.	IntelliJ IDEA, LibreOffice, Photoshop, VLC Media Player.
Aplicaciones Web	Funcionan bajo la arquitectura cliente-servidor. El cliente (navegador web) ejecuta la interfaz (HTML, CSS, JavaScript) mientras que la lógica de negocio y los datos residen en servidores remotos (<i>backend</i>).	Gmail, Spotify Web, plataformas bancarias en línea.
Aplicaciones Móviles	Diseñadas específicamente para dispositivos móviles (teléfonos, tablets) bajo Android o iOS. Pueden ser nativas (Java/Kotlin, Swift) o multiplataforma (Flutter, React Native).	WhatsApp, Instagram, Google Maps móvil.
Servicios / Demonios (Daemons / Background Services)	Procesos que se ejecutan en segundo plano de manera desatendida, sin interacción directa con el usuario, procesando tareas programadas o eventos de sistema.	Servicios de sincronización en la nube, indexadores de búsqueda, servidores web en segundo plano.
Sistemas Embebidos / Empotrados (Embedded)	Software integrado en dispositivos de hardware dedicados con recursos de memoria y procesador muy restringidos y requisitos de tiempo real estricto.	Centralitas de automóviles (ECU), microondas, routers domésticos, domótica IoT.
Servicios Web y APIs REST	Aplicaciones sin interfaz visual que ofrecen servicios de procesamiento o intercambio de datos a otras aplicaciones a través de protocolos de red (JSON, XML).	Pasarelas de pago (Stripe, PayPal), APIs meteorológicas.

2. Evolución Histórica de los Lenguajes de Programación

Para comunicarnos con un procesador electrónico debemos salvar una enorme distancia conceptual: los seres humanos pensamos con lenguaje natural, conceptos abstractos y ambigüedades, mientras que los circuitos integrados de una CPU solo comprenden transistores encendidos o apagados (diferencias de potencial eléctrico representadas como `1` y `0`).

A lo largo de la historia de la computación, los lenguajes han evolucionado a través de cinco generaciones (GL: *Generation Language*), incrementando progresivamente su nivel de abstracción:

Nivel de Abstracción Humana

▲

- | 5GL: Lenguajes basados en Inteligencia Artificial y Reglas Lógicas (Prolog, Mercury)
- | 4GL: Lenguajes Declarativos orientados a Datos y Consultas (SQL, R)
- | 3GL: Lenguajes de Alto Nivel Estructurados y Orientados a Objetos (Java, C, C++, C#, Python)
- | 2GL: Lenguajes Ensambladores (Assembly x86, ARM)
- | 1GL: Código Máquina Binario (Instrucciones en ceros y unos directas para la CPU)

▼

Nivel de Hardware / Circuitos Físicos

2.1 Primera Generación (1GL): Lenguaje Máquina

- Las primeras computadoras de propósito único (como el COLOSSUS en la Segunda Guerra Mundial) se “programaban” modificando manualmente cables e interruptores en paneles físicos.
- Con la llegada de la arquitectura Von Neumann (ENIAC), las instrucciones pasaron a residir en memoria como cadenas binarias de ceros y unos (0 y 1).
- Cada instrucción codifica directamente la operación del microprocesador (*OpCode*) y las direcciones físicas de memoria de los operandos.
- **Inconvenientes:** Extremadamente difícil de programar, nula legibilidad para humanos, facilidad pasmosa para cometer errores irreversibles y dependencia total de la arquitectura del chip: un programa máquina escrito para Intel x86 no funciona en procesadores ARM ni PowerPC.

2.2 Segunda Generación (2GL): Lenguaje Ensamblador (Assembler)

- Sustituye las cadenas binarias por códigos nemotécnicos legibles por humanos (por ejemplo, `MOV AX, 1` para mover un dato a un registro o `ADD AX, BX` para sumar).
- Se introduce el programa **Ensamblador**, encargado de traducir el código nemotécnico a código máquina puro.
- **Inconvenientes:** Aunque mejora la legibilidad, sigue estando estrechamente acoplado a los registros físicos y a la arquitectura interna de un procesador específico. Escribir algoritmos matemáticos complejos requiere cientos de instrucciones de bajo nivel.

2.3 Tercera Generación (3GL): Lenguajes de Alto Nivel

- Nacen a finales de los años 50 con la aparición de **FORTRAN** (1957, considerado el primer lenguaje de alto nivel para cálculo científico), seguido por **LISP** (1958, cálculo simbólico e IA inicial), **COBOL** (1960, aplicaciones de gestión empresarial), **BASIC** (1964, diseñado con fines pedagógicos), **C** (1972, Dennis Ritchie en los Laboratorios Bell) y **Pascal** (Niklaus Wirth, concebido para la enseñanza rigurosa de la algorítmica).
- Más adelante emergen los pioneros de la orientación a objetos: **Simula** (1967, primer lenguaje con noción de clase y objeto), **C++** (Bjarne Stroustrup, 1983), **Java** (James Gosling / Sun Microsystems, 1995) y **C#** (Microsoft, 2000).
- Emplean estructuras lógicas comprensibles, palabras en inglés (`if`, `while`, `class`) y notación matemática estándar (`total = precio * 1.21`).
- **Ventaja colosal:** Son **portables**; un mismo algoritmo puede ejecutarse en múltiples plataformas hardware sin reescribir la lógica, siempre que exista un compilador o intérprete adaptado al sistema de destino.

El mismo “Hola, mundo” a través de la historia

Compara cómo ha cambiado la forma de expresar la misma tarea elemental según la época y el nivel de abstracción del lenguaje:

- ▶ Ensamblador (Intel 8086 con DOS int 21h)
- ▶ FORTRAN (1957)
- ▶ COBOL (1960)
- ▶ C (1972)
- ▶ Pascal
- ▶ Java (1995)
- ▶ Python

Fíjate en la clara tendencia evolutiva: cada nueva generación de lenguajes necesita **menos líneas accesorias** y su código se aproxima **mucho más al lenguaje humano** para alcanzar exactamente el mismo resultado.

2.4 Cuarta Generación (4GL) y Quinta Generación (5GL)

- **4GL (Lenguajes Declarativos):** Se centran en indicar **QUÉ** resultado se desea obtener y no **CÓMO** calcularlo paso a paso. El ejemplo paradigmático es **SQL** (*Structured Query Language*), donde especificamos qué registros deseamos recuperar de una base de datos y el motor optimizador decide la mejor estrategia interna de acceso.
- **5GL (Inteligencia Artificial y Resolución Lógica):** Lenguajes basados en restricciones, hechos y motores de inferencia lógica formal (como **Prolog**), donde el programador suministra las reglas del dominio y el motor resuelve problemas deductivos.

3. Traducción de Programas: Compilación, Interpretación y la Máquina Virtual de Java

Un ordenador no puede ejecutar directamente texto fuente de alto nivel (`.java` , `.c` , `.py`). Debe someterse a un proceso de traducción técnica. Existen tres filosofías principales:

3.1 El Modelo Compilado (Tradicional)

Un programa especializado denominado **compilador** analiza todo el código fuente en bloque antes de su puesta en marcha:

1. **Análisis léxico, sintáctico y semántico:** Comprueba que el código cumpla escrupulosamente las normas gramaticales del lenguaje y no existan incongruencias de tipos.
 2. **Generación de código objeto:** Transforma el código fuente en código máquina intermedio (`.obj` u `.o`).
 3. **Enlazado (*Linking*):** El enlazador (*linker*) combina el código objeto con las bibliotecas estándar del sistema para producir un único fichero binario ejecutable (`.exe` en Windows, binario ELF en Linux).
- **Ventaja:** Máxima velocidad de ejecución nativa en tiempo de producción.
 - **Inconveniente:** Si compilas en Windows para arquitectura x64, el ejecutable generado no funcionará en Linux ni en una CPU ARM sin recompilar todo el proyecto. Lenguajes típicos: C, C++, Rust, Go.

3.2 El Modelo Interpretado

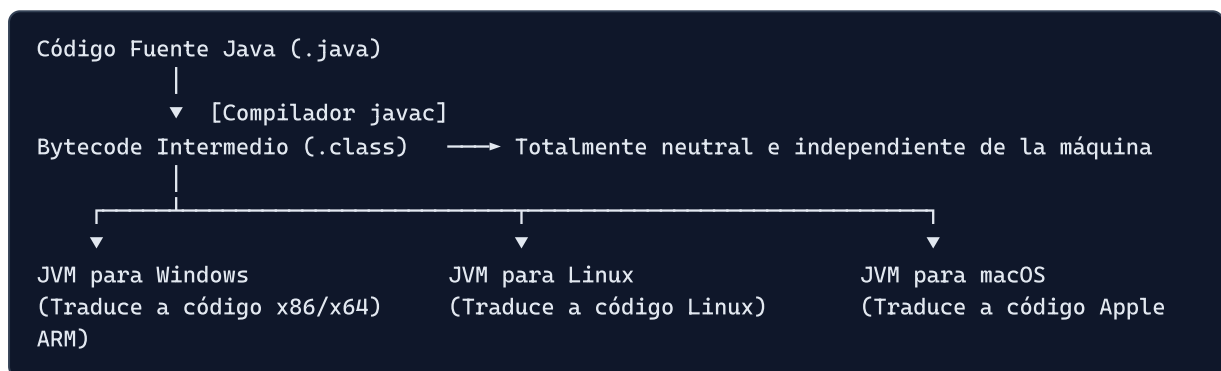
No existe un paso previo de generación de binario ejecutable. Un programa llamado **intérprete** lee el código fuente línea por línea, analiza su sintaxis y la traduce inmediatamente a órdenes del procesador sobre la marcha:

- **Ventaja:** Gran flexibilidad y ciclo interactivo rápido; ideal para scripts y pruebas dinámicas.
- **Inconveniente:** La ejecución es significativamente más lenta, ya que la traducción se realiza en tiempo real en cada ejecución. Si existe un error sintáctico en la línea 500, el programa fallará tras haber ejecutado las 499 anteriores. Lenguajes típicos: JavaScript en navegadores, PHP, Python puro.

3.3 El Modelo Híbrido de Java: Bytecode y la Máquina Virtual (JVM)

Los creadores de Java (James Gosling y Sun Microsystems) revolucionaron la industria del software en 1995 introduciendo una solución intermedia que resuelve el dilema entre velocidad y portabilidad bajo el lema:

"Write Once, Run Anywhere" (Escribe una vez, ejecuta en cualquier parte).



1. El compilador de Java (`javac`) no genera código máquina nativo para un procesador físico, sino **Bytecode**: un conjunto de instrucciones de muy bajo nivel optimizadas para una computadora conceptual e ideal: la **Máquina Virtual de Java (JVM)**.
2. Cada sistema operativo (Windows, Linux, macOS) cuenta con su propia implementación de la JVM.
3. La JVM carga el fichero `.class`, verifica su seguridad y lo ejecuta mediante un motor mixto que combina interpretación inicial con un sofisticado compilador en tiempo de ejecución denominado **JIT (Just-In-Time Compiler)**, el cual detecta los fragmentos de código más ejecutados (*hotspots*) y los convierte en código máquina ultrarrápido idéntico al de un lenguaje compilado nativo.

3.4 Las Herramientas del Desarrollador

Para transformar una idea abstracta en una aplicación plenamente operativa y libre de errores, un programador recurre a un ecosistema de herramientas especializadas:

- **Editor de código fuente:** Aplicación de texto avanzada con coloreado sintáctico, autocompletado inteligente de palabras clave, indentación automática y detección temprana de erratas.
- **Compilador / Intérprete:** El motor de traducción que convierte el código de alto nivel en binario nativo o bytecode.
- **Depurador (Debugger):** Herramienta crucial que permite detener la ejecución en cualquier punto (*breakpoints* o puntos de interrupción), avanzar instrucción por instrucción inspeccionando el contenido exacto de las variables en la memoria RAM y verificar el flujo del programa en tiempo real.
- **Entorno de Desarrollo Integrado (IDE - Integrated Development Environment):** Una suite profesional unificada que aglutina editor, compilador, depurador, herramientas de refactorización de código, gestor de dependencias, terminal integrada y control de versiones (Git) bajo una misma interfaz de trabajo.

Nuestro IDE Oficial: IntelliJ IDEA

En este curso utilizaremos **IntelliJ IDEA** como IDE oficial. Todas las explicaciones prácticas, capturas de pantalla, navegación por menús, atajos de teclado, refactorizaciones y sesiones de depuración (*debugging*) estarán basadas exclusivamente en IntelliJ IDEA. Es una de las herramientas profesionales más potentes y demandadas en la industria del software empresarial internacional.

4. Paradigmas de Programación

Un paradigma de programación es un enfoque metodológico, un estilo fundamental o una filosofía para estructurar y conceptualizar la resolución de problemas mediante software:

4.1 Paradigma Imperativo / Estructurado

Concibe el programa como una serie secuencial de instrucciones que van alterando progresivamente el estado global de la memoria (variables), describiendo al milímetro **CÓMO** resolver el problema.

- **Programación Desestructurada (Histórica):** Predominaba en los primeros lenguajes comerciales. Carecía de bloques formales y abusaba de la instrucción de salto incondicional `GOTO`. Conforme el código crecía, el flujo saltaba erráticamente de una línea a otra, creando el temido **código espagueti**, imposible de mantener o depurar:

```
10 SUMA = 0
20 PRINT "Introduce un número positivo para sumar (negativo para salir):"
30 INPUT N
40 IF N < 0 THEN GOTO 70
50 SUMA = SUMA + N
60 GOTO 20
70 PRINT "La suma acumulada es: " SUMA
```

- **Programación Estructurada:** Se fundamenta en el **Teorema de la Programación Estructurada** (Böhm y Jacopini), que demostró que cualquier algoritmo computable puede resolverse combinando únicamente tres estructuras de control: **secuencia**, **selección** e **iteración**, eliminando definitivamente los saltos `GOTO`:

```
PROGRAM SumadorEstructurado;
VAR
  suma, n: INTEGER;
BEGIN
  suma := 0;
  REPEAT
    WRITE('Introduce un número positivo: ');
    READ(n);
    IF n ≥ 0 THEN
      suma := suma + n;
    UNTIL n < 0;
  WRITELN('La suma acumulada es: ', suma);
END.
```

- **Programación Modular:** Da un paso más allá al descomponer el algoritmo en subprogramas o funciones independientes especializadas (apartado 11).

4.2 Paradigma Orientado a Objetos (POO)

El pilar central de nuestro curso y de Java. En lugar de mantener datos por un lado y funciones sueltas por otro, la POO encapsula **estado** (atributos/variables) y **comportamiento** (métodos/funciones) dentro de entidades coherentes llamadas **Objetos**, que dialogan enviándose mensajes:

```
public class EjemploPOO {
    public static void main(String[] args) {
        // Creamos una instancia u objeto con su propio estado
        Saludo saludoEspanol = new Saludo("Hola");
        saludoEspanol.saludar("María"); // Salida: ¡Hola, María!
    }
}

class Saludo {
    private String prefijo; // Estado interno encapsulado

    public Saludo(String prefijo) {
        this.prefijo = prefijo;
    }

    public void saludar(String nombrePersona) { // Comportamiento
        System.out.println("¡" + this.prefijo + ", " + nombrePersona + "!");
    }
}
```

Sus cuatro pilares fundamentales son: **Abstracción**, **Encapsulamiento**, **Herencia** y **Polimorfismo** (profundizaremos en ellos exhaustivamente en las Unidades 4 y 5).

4.3 Paradigma Declarativo

En contraposición al imperativo, aquí no indicamos *cómo* realizar el cálculo paso a paso, sino **QUÉ** resultado o condición final deseamos obtener; el motor interno se encarga de determinar el plan de ejecución óptimo:

- **Lenguajes de Consulta (SQL):** En lugar de escribir bucles manuales para recorrer tablas y comparar celdas en memoria, declaramos el conjunto deseado:

```
SELECT nombre, departamento, salario
FROM empleados
WHERE salario > 2500.00
ORDER BY salario DESC;
```

- **Programación Funcional:** Trata la computación como la evaluación de funciones matemáticas puras sin estado mutable ni efectos colaterales. A partir de Java 8, Java incorporó expresiones lambda y la API Streams para adoptar este paradigma (Unidad 7).
- **Programación Lógica (Prolog):** Define una base de conocimiento basada en hechos y reglas lógicas, delegando en un motor de inferencia la resolución de hipótesis.

4.4 Paradigma Reactivo

Se basa en flujos de datos asíncronos (*streams*) y en la propagación automática de cambios (patrón observador). Los componentes se suscriben a eventos o emisiones de información y reaccionan en tiempo real ante su llegada (muy utilizado en arquitecturas de alta concurrencia con RxJava o Project Reactor).

● Java es un lenguaje multiparadigma

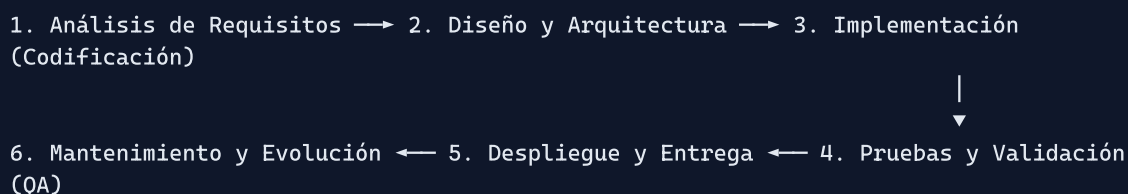
Aunque Java es fundamentalmente orientado a objetos, en sus versiones LTS actuales combina con total fluidez programación imperativa estructurada, componentes declarativos y programación funcional con lambdas y streams.

5. El Ciclo de Vida del Software y Modelado con UML

Crear software comercial de calidad no consiste en sentarse a teclear código improvisado. Requiere seguir un riguroso **ciclo de vida de ingeniería**.

Una analogía muy gráfica para comprender este proceso es **el encargo de una tarta de bodas**:

1. **Análisis de Requisitos:** Hablas en profundidad con los novios: “¿Cuántos invitados vendrán?”, “¿Qué sabores prefieren?”, “¿Hay alérgicos al gluten?”. Es la fase que los programadores novatos más tienden a ignorar, y la que más caro cuesta si se hace mal.
2. **Diseño y Arquitectura:** Dibujas el boceto, calculas los pisos, la estructura de soporte y la combinación de ingredientes antes de tocar la cocina. Es el plano técnico de la solución.
3. **Implementación (Codificación):** Por fin mezclas los ingredientes, horneas y montas la tarta en la cocina siguiendo fielmente la receta y el plano (traducción al lenguaje de programación).
4. **Pruebas y Verificación (QA):** Pruebas una porción de prueba, verificas la textura, la estabilidad y la temperatura antes del evento.
5. **Despliegue y Entrega:** Transportas la tarta y la presentas impecablemente en el salón de bodas ante los comensales.
6. **Mantenimiento:** Te aseguras de que el corte se realice sin incidencias y atiendes peticiones de última hora.



★ Be the Code: La importancia de la documentación y el código limpio

La inmensa mayoría del código que vas a mantener y depurar a lo largo de tu trayectoria profesional **no lo habrás escrito tú**. Escribir código limpio, usar identificadores claros y documentar concienzudamente no es un capricho académico: es pensar con empatía en tu compañero de equipo (o en ti mismo dentro de seis meses, cuando ya no recuerdes por qué pusiste esa condición).

En un proyecto profesional se distinguen dos tipos de documentación:

- **Documentación Interna:** Comentarios dentro del propio código fuente (`//`, `/* */` y comentarios de documentación `javadoc` `/** */`) explicando el *porqué* de las decisiones complejas y las interfaces de los métodos.
- **Documentación Externa:** Manuales de usuario, especificaciones técnicas de requisitos, diagramas de arquitectura UML y guías de despliegue.

5.1 Tipología de Errores en Programación

Durante las fases de codificación y pruebas nos enfrentamos inevitablemente a tres tipos de errores cualitativamente muy distintos:

- **1. Errores de Compilación (Sintácticos):**
 - El código viola las reglas gramaticales del lenguaje (ejemplo: escribir `pritrnln` en vez de `println`, olvidar un punto y coma, o intentar asignar un texto a una variable de tipo entero).
 - El compilador rechaza el archivo y no genera el ejecutable. Son los más fáciles de localizar y corregir porque el propio entorno (IDE) indica la línea exacta del fallo.
- **2. Errores en Tiempo de Ejecución (Runtime Errors / Excepciones):**
 - El código compila con éxito porque sintácticamente es correcto, pero durante la ejecución se produce una situación anómala imprevista que el procesador no puede resolver (ejemplo: dividir un número entre cero, intentar abrir un fichero inexistente en el disco duro o agotar la memoria RAM).
 - Si el programa no gestiona la anomalía mediante control de excepciones (`try-catch`), el sistema operativo aborta bruscamente la aplicación.
- **3. Errores Lógicos o Semánticos (Bugs):**
 - El programa compila sin advertencias y se ejecuta sin abortar, pero **el resultado producido es matemáticamente incorrecto o incoherente con el objetivo planteado** (ejemplo: aplicar una fórmula de descuento sumando en lugar de restar, o plantear mal la condición de un bucle provocando que itere una vez de menos o caiga en un bucle infinito).
 - Son los errores más costosos y difíciles de detectar, y requieren depuración manual (*debugging*) y trazado meticuloso.

5.2 Introducción al Modelado con UML (Unified Modeling Language)

Para plasmar el diseño de un sistema informático de forma universal e inequívoca, los ingenieros recurren a **UML**, un estándar internacional de modelado visual que divide los diagramas en dos grandes familias:

- **Diagramas Estructurales:** Representan la estructura estática del sistema. El más utilizado en programación es el **Diagrama de Clases**, que detalla qué clases componen la aplicación, sus atributos, métodos y cómo se relacionan entre sí (herencia, asociación, agregación y composición).
- **Diagramas de Comportamiento:** Muestran la dinámica y el flujo de ejecución a lo largo del tiempo. Destacan el **Diagrama de Casos de Uso** (qué acciones puede realizar cada actor o usuario), el **Diagrama de Actividades** (flujo paso a paso de un proceso de negocio, muy similar a un diagrama de flujo) y el **Diagrama de Secuencia** (cómo dialogan y se envían mensajes los objetos cronológicamente).

6. Algoritmos: Definición, Propiedades y Buenas Prácticas

Un **algoritmo** es un conjunto ordenado, no ambiguo y finito de instrucciones que describe el procedimiento necesario para transformar los datos de partida en el resultado deseado.

Para visualizarlo, una analogía clásica y tangible es una receta de cocina de la vida diaria:

Freír un huevo perfecto

- **Entrada (Input):** 1 huevo, 50 ml de aceite de oliva, una pizca de sal, sartén, fuente de calor (fuego).
- **Procesamiento:**
 1. Verter el aceite en la sartén.

2. Encender el fuego y calentar la sartén.
3. Cuando el aceite alcance la temperatura adecuada, cascar el huevo y verterlo con cuidado.
4. Bañar la yema con aceite caliente usando una espumadera durante 40 segundos.
5. Cuando la clara esté cuajada, retirar el huevo del fuego y añadir la sal.

- **Salida (Output):** Huevo frito listo para servir.

6.1 Propiedades de un Algoritmo Correcto

En la teoría computacional, distinguimos entre condiciones **obligatorias** (sin las cuales la secuencia no califica como algoritmo) y cualidades **deseables**:

Condiciones Obligatorias:

1. **Resuelve el problema planteado:** Produce la solución exacta y esperada según los requisitos iniciales.
2. **Independencia de la plataforma:** No depende de ningún sistema operativo, arquitectura de procesador ni lenguaje de programación específico.
3. **Precisión:** Cada instrucción debe estar especificada con total exactitud matemática, sin ambigüedades interpretativas.
4. **Finitud:** El algoritmo debe terminar obligatoriamente tras un número finito y computable de pasos; no puede quedar atrapado en un ciclo infinito.
5. **Determinismo (Repetibilidad):** Ante los mismos datos de entrada, el algoritmo debe arrojar exactamente el mismo resultado en todas y cada una de sus ejecuciones.
6. **Entrada y Salida bien delimitadas:** Define con claridad qué datos recibe del exterior y qué información o efecto observable devuelve.

Cualidades Deseables:

- **Válido:** Ausencia de defectos lógicos y robustez probada ante entradas límite o atípicas (*edge cases*).
- **Eficiente:** Resuelve el problema optimizando el tiempo de procesamiento de la CPU y reduciendo el consumo de memoria RAM.
- **Óptimo:** Es la solución más rápida y ligera matemáticamente demostrable de entre todos los algoritmos conocidos para ese problema.

6.2 Reglas de Nombrado de Identificadores (*Naming Conventions*)

Los nombres que asignamos a nuestras variables, constantes y subalgoritmos se denominan **identificadores**. Un código profesional debe ser autoexplicativo (*código limpio*):

- Deben comenzar obligatoriamente por una letra del alfabeto (o guion bajo `_`), nunca por un número.
- No deben contener espacios en blanco, operadores matemáticos (`+`, `-`, `*`, `/`) ni caracteres especiales conflictivos (`!`, `@`, `#`, `$`, `%`).
- Se desaconseja el uso de tildes y ñes para evitar problemas de codificación de caracteres al exportar código entre distintos sistemas operativos.
- **Convención camelCase:** Si el identificador está formado por varias palabras, la primera palabra va en minúsculas y las subsiguientes con su inicial en mayúscula: `salarioNetoMensual`, `horasTrabajadas`, `precioUnitario`.
- **Convención UPPER_SNAKE_CASE para constantes:** Las constantes se escriben íntegramente en mayúsculas separadas por guiones bajos: `PI`, `TASA_IVA_GENERAL`, `MAX_INTENTOS_ACCESO`.

- **Claridad semántica:** Evita a toda costa nombres crípticos o vagos como `x`, `a1`, `temp2`, `cosa`. Usa nombres que describan inequívocamente su función: `contadorIntentos`, `acumuladorGasto`, `distanciaKilometros`.

7. Técnicas de Representación de Algoritmos

Existen tres formas habituales de plasmar un algoritmo: el lenguaje natural humano, los diagramas de flujo y el pseudocódigo.

7.1 Lenguaje Natural: El Peligro de la Ambigüedad

Si intentamos describir un algoritmo complejo en español común, tropezamos inmediatamente con la polisemia, las interpretaciones subjetivas y la falta de rigor formal:

“Pela las patatas, córtalas finas y fríelas en aceite caliente hasta que doren. Añade huevo y cuájalo.”

¿Cuánto de finas? ¿A qué temperatura exacta debe estar el aceite? ¿Cuántos huevos? El lenguaje natural es excelente para la literatura, pero pésimo para dar órdenes a una máquina.

7.2 Diagramas de Flujo (Ordinogramas) según la Norma ANSI/ISO

Un diagrama de flujo representa gráficamente la secuencia lógica de un algoritmo empleando formas geométricas estandarizadas por la norma **ANSI X3.5**:

Símbolo	Nombre ANSI	Función y Significado
 (Óvalo / Rectángulo redondeado)	Terminal	Representa el punto de arranque (Inicio) o de conclusión (Fin) del algoritmo o subproceso.
 (Rectángulo)	Proceso	Operación interna que modifica datos en memoria: cálculos aritméticos, asignaciones de variables (<code>total ← base × 1.21</code>).
 (Paralelogramo)	Entrada / Salida	Operación de lectura de información externa (<code>Leer variable</code>) o de visualización de datos al usuario (<code>Escribir resultado</code>).
 (Rombo)	Decisión	Evalúa una condición lógica booleana. Cuenta obligatoriamente con dos o más caminos de salida etiquetados (<code>Verdadero</code> / <code>Falso</code> o <code>Sí</code> / <code>No</code>).
 (Círculo pequeño)	Conector Interno	Enlaza partes del diagrama dentro de la misma página para evitar un cruce caótico de líneas.
 (Pentágono invertido)	Conector de Página	Conecta flujos entre páginas o láminas diferentes de un documento.
 (Líneas de flujo)	Líneas de Flujo	Indican la dirección obligatoria del flujo de ejecución. Nunca deben cruzarse entre sí sin conector y no pueden apuntar al vacío.

7.3 Pseudocódigo Estructurado con PSeInt

El pseudocódigo es un lenguaje intermedio artificial que adopta la estructura formal de un lenguaje de programación real pero utilizando términos en lenguaje humano comprensible.

En nuestro curso nos basamos en las convenciones formales de **PSeInt** con perfil estricto:

- Todo algoritmo comienza con la palabra reservada `Algoritmo NombreAlgoritmo` y finaliza con `FinAlgoritmo`.
- Toda variable debe ser declarada explícitamente al inicio mediante `Definir identificador Como TipoDato`.
- La asignación se denota con la flecha direccional `←` para no confundirla con la comparación de igualdad.

8. Elementos del Pseudocódigo: Tipos, Operadores y Lógica Booleana

8.1 Tipos de Datos Primitivos

- **Entero:** Representa números enteros sin componente decimal, tanto positivos como negativos (`-10`, `0`, `450`).
- **Real:** Representa números continuos con parte fraccionaria o decimal (`3.14159`, `-0.005`, `12.0`).
- **Carácter:** Almacena un único símbolo tipográfico (letra, dígito o signo de puntuación) delimitado por comillas simples (`'A'`, `'9'`, `'#'`).
- **Cadena:** Secuencia ordenada de caracteres delimitada por comillas dobles (`"Programación Java"`, `"IES Serra Perenxisa"`).
- **Lógico (Booleano):** Tipo algebraico binario que solo puede admitir uno de dos valores posibles: `Verdadero` o `Falso`.

8.2 Operadores Aritméticos y División

- **Potencia:** `^` (ejemplo: `2 ^ 3 = 8`).
- **Multiplicación:** `*` o `×`.
- **División real:** `/` (produce siempre un resultado con decimales: `7.0 / 2.0 = 3.5`).
- **División entera truncada:** `/` sobre operandos enteros (`7 / 2 = 3`).
- **Módulo / Resto:** `MOD` o `%`:
 - La operación `A MOD B` calcula el residuo que sobra tras realizar la división entera entre `A` y `B`.
 - *Ejemplo:* Si dividimos `17` entre `5`, el cociente entero es `3` ($5 \times 3 = 15$) y el resto que sobra es `2`. Por tanto: `17 MOD 5 = 2`.
 - El operador módulo es crucial en programación para detectar números pares o impares (`n MOD 2 = 0`), comprobar divisibilidad o reiniciar contadores circulares.
- **Suma (+) y Resta (-).**

8.3 Operadores Relacionales (Comparación)

Permiten comparar dos valores del mismo tipo de dato, produciendo siempre un resultado de tipo `Lógico`:

- Igual a: `=` (o `==` en Java).
- Distinto de: `≠` o `<>` (o `≠` en Java).
- Menor que: `<` | Menor o igual que: `≤` o `≤`.
- Mayor que: `>` | Mayor o igual que: `≥` o `≥`.

8.4 Operadores Lógicos y Álgebra de Boole

Permiten combinar condiciones lógicas complejas:

- **Negación (NO / NOT):** Operador unario que invierte el valor de verdad.

- **Conjunción (Y / AND)**: Devuelve **Verdadero** únicamente si ambas expresiones operadas son verdaderas simultáneamente.
- **Disyunción Inclusiva (O / OR)**: Devuelve **Verdadero** si al menos una de las expresiones operadas es verdadera.
- **Disyunción Exclusiva (OEX / XOR)**: Devuelve **Verdadero** si una expresión es verdadera y la otra es falsa, pero no si ambas lo son a la vez.

Tablas de Verdad Completas

Proposición A	Proposición B	NO A	A Y B	A O B	A XOR B
Falso	Falso	Verdadero	Falso	Falso	Falso
Falso	Verdadero	Verdadero	Falso	Verdadero	Verdadero
Verdadero	Falso	Falso	Falso	Verdadero	Verdadero
Verdadero	Verdadero	Falso	Verdadero	Verdadero	Falso

Un caso clásico que engaña: La hora de tutoría

Imaginemos la siguiente regla de un centro educativo: “La tutoría del profesor es los lunes y martes a las 16 horas”. ¿Cuál de las siguientes expresiones lógicas traduce fielmente la norma?

- **Opción A:** `dia = "lunes" O dia = "martes" Y hora = 16`
- **Opción B:** `(dia = "lunes" O dia = "martes") Y hora = 16`

Aunque en el lenguaje cotidiano suenen parecidas, en programación son radicalmente distintas. Como el operador lógico **Y** tiene mayor precedencia que **O** (exactamente igual que la multiplicación prima sobre la suma), la Opción A se evalúa como:

```
dia = "lunes" O (dia = "martes" Y hora = 16)
```

¡Lo que significa que la condición sería válida **a cualquier hora de los lunes**! La opción correcta es la **Opción B**, usando paréntesis explícitos para agrupar los días lectivos. Ante la menor duda en una expresión lógica compuesta, utiliza siempre paréntesis.

Leyes de De Morgan

Son dos reglas lógicas fundamentales para simplificar condiciones complejas y diseñar pruebas unitarias:

1. `NO (A Y B)` es lógicamente equivalente a `(NO A) O (NO B)`.
2. `NO (A O B)` es lógicamente equivalente a `(NO A) Y (NO B)`.

8.5 Jerarquía Universal de Precedencia

Cuando una expresión contiene múltiples operadores, el computador los evalúa siguiendo un estricto orden matemático:

1. **Paréntesis:** Expresiones delimitadas por `( ... )`, evaluadas desde el par más interno hacia el exterior.
2. **Potenciación:** `^`.
3. **Operadores unarios:** Signo negativo unario (`-x`) y negación lógica (`NO`).
4. **Multiplicación, División y Módulo:** `*`, `/`, `MOD` (se evalúan de izquierda a derecha en caso de empate).
5. **Suma y Resta:** `+`, `-` (de izquierda a derecha).

6. **Operadores Relacionales:** `<`, `>`, `≤`, `≥`, `=`, `≠`.

7. **Conjunción Lógica:** `Y`.

8. **Disyunción Lógica:** `O` y `XOR`.

¡Cuidado con la precedencia!

La expresión `9 + 6 / 3` evalúa a **11**, no a **5**, porque la división tiene prioridad estricta sobre la suma. Si deseamos sumar primero, es indispensable forzarlo con paréntesis: `(9 + 6) / 3 = 5`. Emplear paréntesis siempre que tengas dudas cuesta una sola pulsación de tecla y previene fallos lógicos muy difíciles de rastrear.

9. Estructuras de Control Algorítmicas en Detalle

9.1 Estructura Secuencial

El flujo de control avanza linealmente de arriba a abajo, ejecutando una instrucción detrás de otra:

```
Algoritmo FacturacionSimple
  Definir baseImponible, porcentajeIva, importeIva, totalPagar Como Real

  Escribir "Introduce el importe de la base imponible en euros:"
  Leer baseImponible

  porcentajeIva ← 21.0
  importeIva ← baseImponible × (porcentajeIva / 100.0)
  totalPagar ← baseImponible + importeIva

  Escribir "Base imponible: ", baseImponible, " €"
  Escribir "IVA (21%):      ", importeIva, " €"
  Escribir "Total factura: ", totalPagar, " €"
FinAlgoritmo
```

9.2 Estructuras de Selección (Condicionales)

Selección Simple y Doble (`Si - Entonces - SiNo`)

Evalúa una condición booleana para bifurcar la ejecución en dos caminos alternativos y excluyentes:

```
Algoritmo DescuentoComercial
  Definir totalCompra, totalConDescuento, rebaja Como Real

  Escribir "Introduce el importe total de la compra:"
  Leer totalCompra

  Si totalCompra > 100.0 Entonces
    rebaja ← totalCompra × 0.10 // 10% de descuento para compras superiores a 100 €
    totalConDescuento ← totalCompra - rebaja
    Escribir "¡Enhorabuena! Tienes un descuento de: ", rebaja, " €"
    Escribir "Importe final a pagar: ", totalConDescuento, " €"
  SiNo
    Escribir "No se aplica descuento (compra ≤ 100 €). Total a pagar: ",
    totalCompra, " €"
  FinSi
FinAlgoritmo
```


Selección Anidada

Cuando la toma de decisiones requiere evaluar múltiples escenarios escalonados:

```
Algoritmo ClasificacionClimatica
  Definir temperatura Como Real
  Escribir "Introduce la temperatura en °C:"
  Leer temperatura

  Si temperatura < 0.0 Entonces
    Escribir "Estado: Congelación / Nieve"
  SiNo
    Si temperatura < 18.0 Entonces
      Escribir "Estado: Ambiente frío"
    SiNo
      Si temperatura < 28.0 Entonces
        Escribir "Estado: Ambiente templado / Agradable"
      SiNo
        Escribir "Estado: Ambiente caluroso / Canícula"
      FinSi
    FinSi
  FinSi
FinSi
FinAlgoritmo
```

Selección Múltiple (**Segun - Hacer** / **Switch**)

Especialmente indicada cuando una variable entera o de carácter se contrasta frente a un catálogo discreto de valores posibles (como los menús interactivos):

```

Algoritmo MenuOperaciones
  Definir opcion Como Entero
  Definir num1, num2, resultado Como Real

  Escribir "===== "
  Escribir "  CALCULADORA ARITMÉTICA  "
  Escribir "===== "
  Escribir "1. Sumar"
  Escribir "2. Restar"
  Escribir "3. Multiplicar"
  Escribir "4. Dividir"
  Escribir "Selecciona una opción (1-4):"
  Leer opcion

  Escribir "Introduce el primer número:"
  Leer num1
  Escribir "Introduce el segundo número:"
  Leer num2

  Segun opcion Hacer
    1:
      resultado ← num1 + num2
      Escribir "Resultado de la suma: ", resultado
    2:
      resultado ← num1 - num2
      Escribir "Resultado de la resta: ", resultado
    3:
      resultado ← num1 × num2
      Escribir "Resultado de la multiplicación: ", resultado
    4:
      Si num2 ≠ 0 Entonces
        resultado ← num1 / num2
        Escribir "Resultado de la división: ", resultado
      SiNo
        Escribir "Error matemático: No es posible dividir entre cero."
      FinSi
  De Otro Modo:
    Escribir "Opción no reconocida en el menú."
  FinSegun
FinAlgoritmo

```

9.3 Estructuras Iterativas (Bucles)

Los bucles permiten repetir un bloque de instrucciones múltiples veces. En programación estructurada existen tres modalidades fundamentales:

Bucle **Mientras** (Bucle con Pre-condición)

- Evalúa la condición lógica **antes** de ejecutar cada iteración.
- **Comportamiento clave:** Si la condición se evalúa como **Falsa** desde el inicio, el cuerpo del bucle **no se ejecutará ninguna vez (0 veces)**.
- Requiere que dentro del bucle exista al menos una instrucción que modifique las variables de la condición; de lo contrario, caerá en un **bucle infinito**.

```

Algoritmo MediaCalificaciones
  Definir sumaNotas, notaActual, media Como Real
  Definir totalAlumnos, contador Como Entero

  Escribir "¿Cuántos alumnos hay en el aula?"
  Leer totalAlumnos

  sumaNotas ← 0.0
  contador ← 1

  Mientras contador ≤ totalAlumnos Hacer
    Escribir "Introduce la nota del alumno número ", contador, ":"
    Leer notaActual
    sumaNotas ← sumaNotas + notaActual
    contador ← contador + 1 // Instrucción de avance indispensable
  FinMientras

  Si totalAlumnos > 0 Entonces
    media ← sumaNotas / totalAlumnos
    Escribir "La media de la clase es: ", media
  SiNo
    Escribir "No se han introducido alumnos."
  FinSi
FinAlgoritmo

```

Bucle **Repetir – Hasta Que** (Bucle con Post-condición)

- Ejecuta primero el bloque de instrucciones y comprueba la condición al final.
- **Comportamiento clave:** El cuerpo del bucle se ejecutará **como mínimo una vez (≥ 1 vez)**, independientemente del estado de la condición.
- En PSeInt, la iteración continúa mientras la condición sea falsa y **se detiene en el instante en que la condición sea verdadera**.
- Es el patrón canónico para la **validación y saneamiento de entradas de usuario**:

```

Algoritmo ValidacionEdadSegura
  Definir edad Como Entero

  Repetir
    Escribir "Introduce una edad válida para el registro laboral [16 - 65 años]:"
    Leer edad
    Si edad < 16 O edad > 65 Entonces
      Escribir "Error: La edad no cumple los criterios legales. Inténtalo de nuevo."
    FinSi
  Hasta Que edad ≥ 16 Y edad ≤ 65

  Escribir "Edad validada y registrada correctamente: ", edad, " años."
FinAlgoritmo

```

Bucle **Para** (Bucle de Contador Automático)

- Se utiliza cuando el número exacto de iteraciones se conoce de antemano antes de entrar al bucle.
- Gestiona automáticamente la inicialización de la variable de control, la comprobación del límite y el incremento (o decremento si el paso es negativo):

```

Algoritmo CuentaAtrasYPaso
  Definir i Como Entero

  Escribir "--- Conteo decreciente de dos en dos ---"
  Para i ← 10 Hasta 0 Con Paso -2 Hacer
    Escribir "Valor actual de i: ", i
  FinPara
  Escribir "¡Final del bucle Para!"
FinAlgoritmo

```

Bucles Anidados

Un bucle puede alojarse dentro de otro. En cada iteración completa del bucle exterior, el bucle interior ejecutará todas sus iteraciones desde el principio hasta el final:

```

Algoritmo DibujarRectanguloAsteriscos
  Definir filas, columnas, f, c Como Entero
  filas ← 4
  columnas ← 6

  Para f ← 1 Hasta filas Con Paso 1 Hacer
    Para c ← 1 Hasta columnas Con Paso 1 Hacer
      Escribir Sin Bajar "*"
    FinPara
    Escribir "" // Salto de línea al terminar cada fila
  FinPara
FinAlgoritmo

```

10. ★ Be the Code: Metodología Formal de la Traza

En programación, "**Be the Code**" es la disciplina de simular mentalmente el papel de la CPU, ejecutando el algoritmo paso a paso sobre una libreta física. La tabla de traza (también llamada prueba de escritorio o dry run) es la herramienta más potente para erradicar errores de lógica sin necesidad de adivinar.

Analicemos minuciosamente un algoritmo de búsqueda y cálculo de divisores: el **Algoritmo de Euclides para el Máximo Común Divisor (MCD)** con dos números de entrada: **A = 30** y **B = 12**.

```

Algoritmo TrazaEuclides
  Definir a, b, resto Como Entero
  a ← 30
  b ← 12

  Mientras b ≠ 0 Hacer
    resto ← a MOD b
    a ← b
    b ← resto
  FinMientras

  Escribir "El MCD obtenido es: ", a
FinAlgoritmo

```

Tabla de Seguimiento de Estados de Memoria

Paso	Línea de Código	a	b	resto	Evaluación de la Condición	Efecto en Memoria / Salida
1	<code>a ← 30</code>	30	<i>indefinido</i>	<i>indefinido</i>	—	Asignación inicial de variable a
2	<code>b ← 12</code>	30	12	<i>indefinido</i>	—	Asignación inicial de variable b
3	<code>Mientras b ≠ 0</code>	30	12	<i>indefinido</i>	<code>12 ≠ 0</code> (Verdadero)	La condición se cumple: entramos a la Iteración 1
4	<code>resto ← a MOD b</code>	30	12	6	—	<code>30 MOD 12 = 6</code> (porque $30 = 12 \times 2 + 6$)
5	<code>a ← b</code>	12	12	6	—	a adopta el valor de b (pasa a valer 12)
6	<code>b ← resto</code>	12	6	6	—	b adopta el valor de resto (pasa a valer 6)
7	<code>Mientras b ≠ 0</code>	12	6	6	<code>6 ≠ 0</code> (Verdadero)	La condición se cumple: entramos a la Iteración 2
8	<code>resto ← a MOD b</code>	12	6	0	—	<code>12 MOD 6 = 0</code> (división exacta: $12 = 6 \times 2 + 0$)
9	<code>a ← b</code>	6	6	0	—	a adopta el valor de b (pasa a valer 6)
10	<code>b ← resto</code>	6	0	0	—	b adopta el valor de resto (pasa a valer 0)
11	<code>Mientras b ≠ 0</code>	6	0	0	<code>0 ≠ 0</code> (Falso)	La condición falla: sale inmediatamente del bucle
12	<code>Escribir "El MCD ... ", a</code>	6	0	0	—	Emisión de salida: "El MCD obtenido es: 6"

💡 Consejo para depurar: La prueba de escritorio

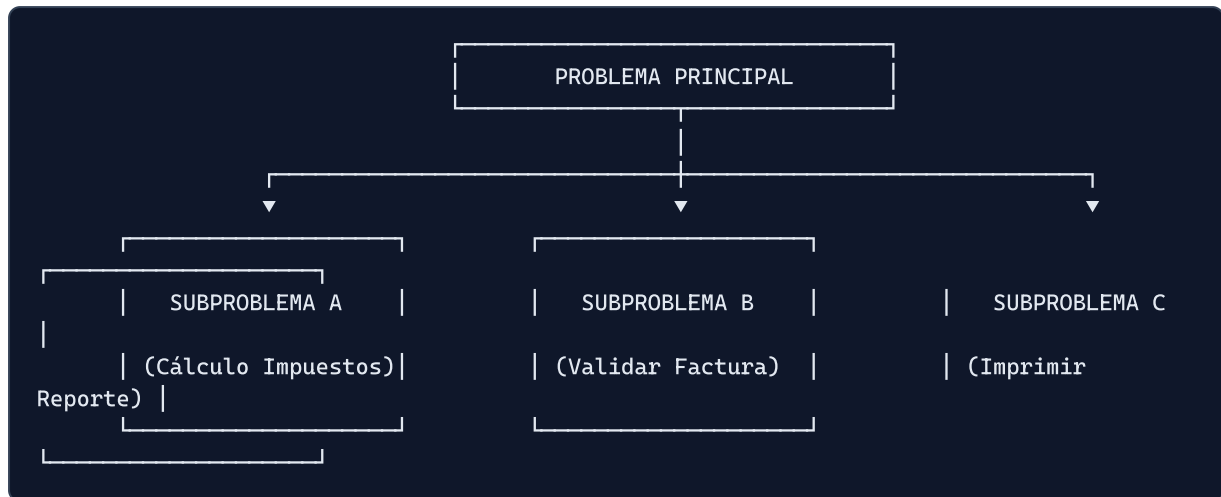
Cuando un programa produzca un resultado inesperado durante este curso, no cambies líneas impulsivamente en el IDE. Coge papel y bolígrafo, dibuja una tabla con una columna para cada variable y evalúa cada línea exactamente como lo haría el procesador. Detectarás la discrepancia en segundos.

11. Programación Modular y Subalgoritmos

Conforme un problema real crece en envergadura, escribir cientos de líneas en un único bloque lineal desemboca en un código monolítico, redundante y frágil.

Imagina una tarea de 20 instrucciones que necesitas repetir en 7 lugares distintos de tu aplicación. Copiar y pegar esas 20 líneas siete veces puede parecer rápido al principio, pero es una pésima práctica de ingeniería: el código se vuelve kilométrico, ilegible y, si dentro de dos meses descubres un fallo o cambia la normativa fiscal, tendrás que localizar y modificar los 7 puntos idénticos (arriesgándote a olvidar uno).

Para solucionarlo aplicamos el principio de la ingeniería clásica **“Divide y Vencerás”** (*Top-Down Design* o descomposición modular descendente):



El error más habitual al empezar

Cuando se empieza a programar, la tentación inmediata es escribirlo todo en un único bloque continuo enorme. Aunque “funcione” en ejercicios sencillos de diez líneas, se vuelve inmanejable y propenso a catástrofes en proyectos reales. Acostúmbrate desde el primer día a pensar en subalgoritmos pequeños, cada uno con una única responsabilidad clara y bien delimitada.

11.1 Procedimientos vs Funciones

Un subalgoritmo es un módulo de código independiente dotado de un nombre propio que resuelve una tarea especializada:

- **Procedimiento:** Subalgoritmo que ejecuta una serie de acciones o efectos secundarios (por ejemplo, limpiar la pantalla, imprimir un informe formateado o escribir en disco) sin retornar formalmente un valor de resultado a la instrucción llamante.
- **Función:** Subalgoritmo que recibe cero o más datos de entrada, procesa un cálculo y **devuelve obligatoriamente un único valor de resultado** al punto donde fue invocada.

```
// DEFINICIÓN DE UNA FUNCIÓN MATEMÁTICA
Funcion retorno ← CalcularAreaCirculo(radio)
    Definir retorno, constantePi Como Real
    constantePi ← 3.14159265
    retorno ← constantePi × (radio × radio)
FinFuncion

// DEFINICIÓN DE UN PROCEDIMIENTO VISUAL
SubProceso ImprimirCabecera(titulo)
    Escribir "===== "
    Escribir " ", titulo
    Escribir "===== "
FinSubProceso

// ALGORITMO PRINCIPAL CONSUMIDOR
Algoritmo GeometriaModular
    Definir r, area Como Real

    ImprimirCabecera("SISTEMA DE CÁLCULO GEOMÉTRICO")
    r ← 5.0
    area ← CalcularAreaCirculo(r) // Llamada a la función
    Escribir "El área para radio ", r, " es: ", area
FinAlgoritmo
```

11.2 Parámetros Formales vs Parámetros Actuales (Argumentos)

- **Parámetros Formales:** Son las variables ficticias declaradas en la cabecera del subalgoritmo que definen qué datos necesita recibir para operar (en el ejemplo anterior: `radio` o `titulo`).
- **Parámetros Actuales (o Argumentos):** Son los valores, expresiones o variables reales que el programa principal transfiere al subalgoritmo en el momento exacto de la llamada (en el ejemplo: el valor `r` o el texto `"SISTEMA ... "`).

11.3 Mecanismos de Paso de Parámetros

Existen dos formas fundamentales de transmitir información a un subalgoritmo:

1. Paso por Valor:

- El subalgoritmo recibe una **copia exacta e independiente** del contenido de la variable original.
- Cualquier modificación que el subalgoritmo realice sobre su parámetro formal **NO alterará** el valor de la variable original del llamante.
- *Ejemplo:* Si le dejas una fotocopia de tus apuntes a un compañero y él pinta sobre la fotocopia, tu folio original permanece inalterado.

2. Paso por Referencia:

- El subalgoritmo no recibe una copia, sino la **dirección de memoria física** donde reside la variable original.
- Cualquier alteración que sufra el parámetro dentro del subalgoritmo **modificará directamente y de forma permanente la variable original** del programa principal.

⚠ Advertencia crucial: El paso de parámetros en Java

En Java, todos los parámetros se pasan estrictamente **por valor**. Sin embargo, cuando pasamos un objeto o un array, el valor copiado es la propia referencia (puntero) a la memoria heap, lo cual genera un comportamiento equivalente a la referencia sobre los atributos internos del objeto. Analizaremos este mecanismo crucial en la Unidad 4.

11.4 Ámbito y Visibilidad de Variables (Scope)

- **Variables Locales:** Declaradas dentro de un subalgoritmo o bloque específico. Solo existen en la memoria mientras dicho bloque se está ejecutando. Al terminar la función, se destruyen. Ninguna otra parte del programa puede acceder a ellas.
- **Variables Globales:** Declaradas en el ámbito raíz del programa, accesibles y modificables desde cualquier subalgoritmo en cualquier momento.
- **Buena práctica de ingeniería:** Debe evitarse el uso de variables globales. Provocan dependencias ocultas, acoplamiento dañino y dificultan enormemente la detección de fallos. La información debe fluir siempre a través de parámetros y valores de retorno.

11.5 Cohesión y Acoplamiento

El diseño de software modular se guía por dos métricas de calidad esenciales:

- **Alta Cohesión (Deseable):** Cada módulo, subalgoritmo o función debe tener una única responsabilidad bien delimitada y cumplirla a la perfección.
- **Bajo Acoplamiento (Deseable):** Los módulos deben ser lo más independientes entre sí posible, comunicándose únicamente mediante interfaces públicas claras (parámetros y retornos), minimizando las dependencias compartidas.

12. ¡No Hay Preguntas Tontas! 💡**¿Por qué estudiamos pseudocódigo y diagramas de flujo si en el trabajo nos van a pedir Java, C# o Python?**

Cuando aprendes a conducir, primero aprendes las normas viales, la interpretación de señales y la dinámica de anticipación del tráfico. Si intentaras aprender todo eso subido a un coche deportivo de competición a 200 km/h bajo la lluvia, te estrellarías en la primera curva sin saber si falló el embrague o si tomaste mal la rotonda.

Java es un lenguaje fuertemente tipado con una sintaxis rigurosa que exige clases, métodos estáticos, puntos y comas, paquetes y llaves `{}`. Si cometes un error siendo principiante, es muy difícil discernir si tu fallo es de lógica matemática o un simple despiste gramatical. El pseudocódigo aísla tu cerebro de la sintaxis y te enseña a pensar con lógica algorítmica pura. Un buen algoritmo en pseudocódigo se traduce a Java en diez minutos; un mal algoritmo no funcionará en ningún lenguaje del planeta.

¿Por qué `17 / 4` puede dar `4` en lugar de `4.25`?

*En los ordenadores, los tipos de datos condicionan el tipo de operación aritmética. Si ambos operandos son de tipo **Entero**, la máquina aplica **división entera truncada**, descartando la fracción decimal: `17 / 4 = 4`. Para obtener el resultado decimal exacto (`4.25`), al menos uno de los operandos debe declararse explícitamente como número **Real**.*

($17.0 / 4$ o $17 / 4.0$). El residuo exacto que se descartó en la división entera se recupera mediante la operación módulo: $17 \text{ MOD } 4 = 1$.

¿Cuál es la diferencia fundamental entre el operador `=` y el operador `←` o `==`?

En matemáticas, el símbolo `=` representa una equivalencia simétrica y pasiva: si decimos que $x = 5$, estamos afirmando un hecho invariable. En programación, la **asignación** (`←` o `=` en Java) es una acción direccional y destructiva: significa "toma el valor calculado a la derecha y guárdalo dentro de la celda de memoria identificada a la izquierda, sobrescribiendo cualquier dato previo". Por eso una instrucción como $x \leftarrow x + 1$ tiene perfecto sentido en informática (incrementar x), mientras que en matemáticas sería una aberración insoluble ($x = x + 1 \rightarrow 0 = 1$). Para contrastar si dos variables son iguales sin modificarlas, se emplea la comparación de igualdad (`=` en pseudocódigo o `==` en Java).

¿Por qué los bucles **Mientras** pueden no ejecutarse ninguna vez y los **Repetir** siempre se ejecutan al menos una?

Porque el bucle **Mientras** es de **pre-condición**: comprueba la puerta de entrada antes de franquear el paso. Si la condición es falsa antes de empezar, el ordenador salta el bloque completo. Por el contrario, el bucle **Repetir** (o **do-while** en Java) es de **post-condición**: entra al cuerpo del bucle, ejecuta las instrucciones y solo cuando llega al final evalúa si debe repetir o salir. Por esta razón, **Repetir** garantiza siempre un mínimo de una ejecución.

¿Repetir ... Hasta Que y Mientras no son lo mismo pero al revés?

Casi, pero con una diferencia esencial: **cuándo se evalúa la condición**.

- **Mientras** pregunta primero y actúa después (puede que no actúe ninguna vez si la condición inicial ya es falsa).
- **Repetir ... Hasta Que** actúa primero y pregunta después (se ejecuta siempre como mínimo una vez).

Además, en pseudocódigo estándar, **Mientras** continúa iterando mientras su expresión evaluada sea **Verdadera**, mientras que **Repetir ... Hasta Que** se mantiene iterando mientras sea **Falsa** y se detiene en el momento justo en que la condición se vuelve **Verdadera**. Por lo tanto, para transformar un bucle en el otro no basta con mover la sentencia: ¡hay que invertir toda la lógica booleana de la expresión!

13. Resumen de la Unidad y Enlace con Java

En esta extensa unidad has adquirido los cimientos sólidos de la ingeniería del software:

- Cómo procesa información un computador bajo el modelo de **caja negra**.
- La evolución de los lenguajes de programación desde el código binario 1GL hasta los lenguajes declarativos 4GL y lógicos 5GL.
- Las diferencias operativas entre **compilación**, **interpretación** y el innovador modelo híbrido de **Java** (**Bytecode + JVM + JIT**).
- El ecosistema de herramientas: editores, depuradores, compiladores y el IDE oficial **IntelliJ IDEA**.
- Los cuatro grandes paradigmas computacionales: imperativo estructurado, orientado a objetos, funcional y reactivo.
- El **ciclo de vida del software** (analogía de la tarta de bodas), la importancia de la documentación interna y externa, y la tipología de errores (sintácticos, de ejecución y lógicos).

- Las propiedades formales de un algoritmo: obligatorias (precisión, determinismo, finitud) y deseables (validez, eficiencia, optimización).
- La simbología oficial **ANSI/ISO** para construir diagramas de flujo claros.
- Los tipos primitivos, operadores aritméticos, relacionales y lógicos con sus tablas de verdad y leyes de De Morgan.
- Las tres estructuras de control canónicas: secuencia, selección (**Si-Entonces** , **Segun**) e iteración (**Mientras** , **Repetir** , **Para**).
- La técnica de trazado manual **Be the Code** para depurar algoritmos sobre papel.
- El diseño modular mediante **procedimientos**, **funciones**, paso de parámetros (por valor vs por referencia) y visibilidad de variables.

En la **Unidad 2**, nos trasladaremos a nuestro entorno de desarrollo oficial **IntelliJ IDEA**, configuraremos el Java Development Kit (**JDK**) y traduciremos todos estos fundamentos a la sintaxis real de **Java LTS**.

14. Relación Curricular: Resultados de Aprendizaje y Criterios de Evaluación

Esta unidad cubre los contenidos troncales del currículo oficial del módulo **0485 - Programación (DAM)**:

Resultado de Aprendizaje (RA)	Criterios de Evaluación Oficiales Cubiertos
RA1. Reconoce la estructura de un programa informático, identificando y relacionando los elementos propios del lenguaje de programación utilizado.	CE 1.a) Se ha reconocido la estructura de un programa informático. CE 1.c) Se han utilizado entornos integrados de desarrollo (IntelliJ IDEA). CE 1.i) Se han introducido comentarios explicativos y documentación adecuada en el código fuente.

15. Boletines de Ejercicios y Retos Prácticos

Pon a prueba tu pensamiento algorítmico y consolida lo aprendido con el repertorio de actividades prácticas de la Unidad 1:

[🟢 Boletín Inicial \(Propuestos\)](#)
[✅ Boletín Inicial \(Resueltos\)](#)
[⭐ Boletín Intermedio \(Propuestos\)](#)
[✅ Boletín Intermedio \(Resueltos\)](#)
[🔥 Retos y Plataformas Externas](#)

📄 Versiones en PDF para imprimir

- [📄 Boletín Inicial \(PDF\)](#) · [Con Soluciones \(PDF\)](#)
- [📄 Boletín Intermedio \(PDF\)](#) · [Con Soluciones \(PDF\)](#)
- [📄 Retos y Extras \(PDF\)](#)